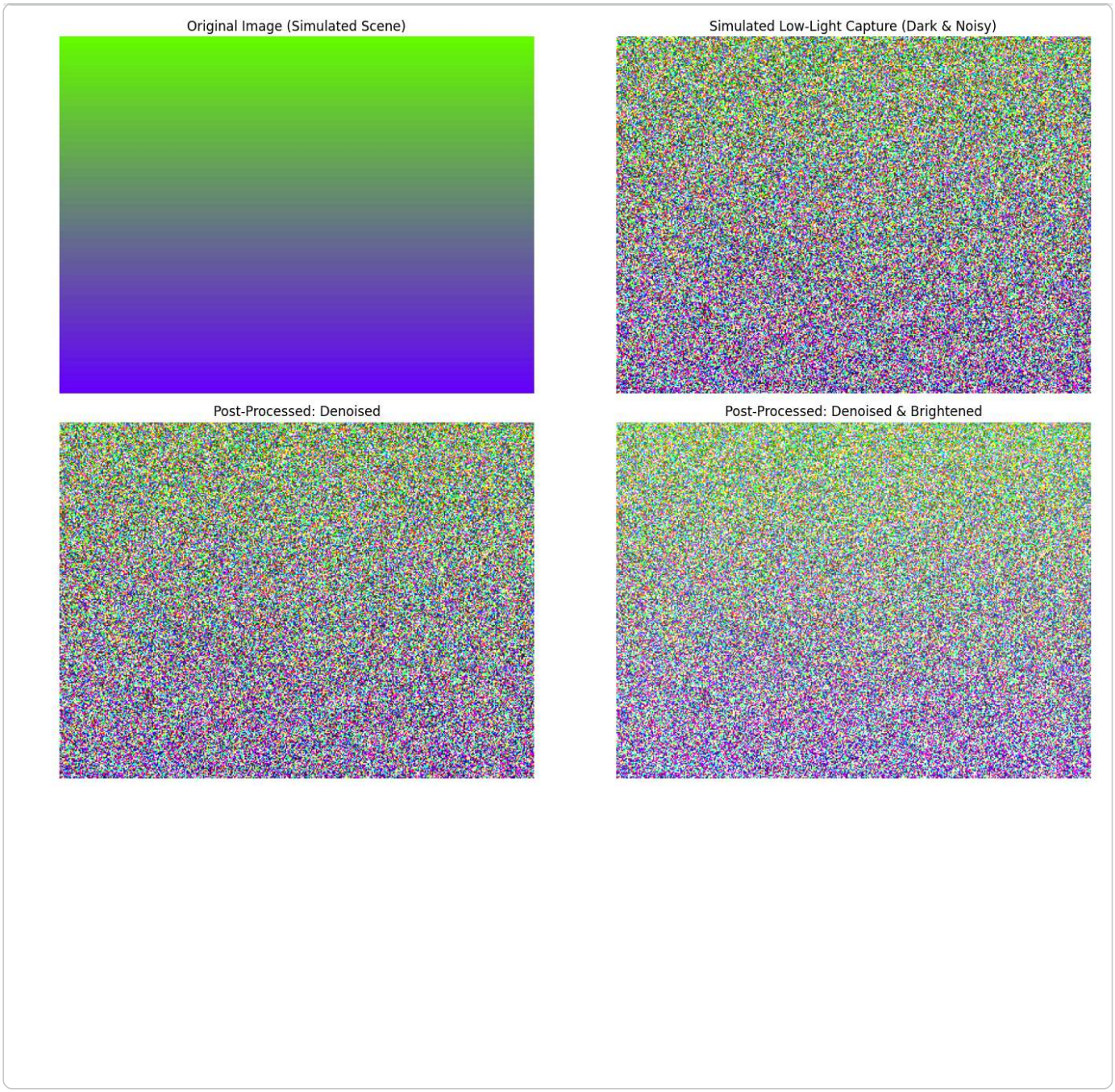
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Create a dummy image to simulate a scene
# In a real scenario, you would load an actual image:
# image = cv2.imread('your_scene_image.jpg')
# For demonstration, let's create a simple gradient image
width, height = 400, 300
img = np.zeros((height, width, 3), dtype=np.uint8)
for i in range(height):
    img[i, :, 0] = int(255 * (i / height)) # Blue gradient
    img[i, :, 1] = int(255 * (1 - i / height)) # Green gradient
    img[i, :, 2] = 100 # Red component

# --- Simulate Low-Light Acquisition --- #

# 1. Reduce overall brightness
low_light_img = cv2.convertScaleAbs(img, alpha=0.3, beta=0) # alpha < 1 reduces brightness

# 2. Add significant Gaussian noise (simulates high ISO/low SNR)
mean = 0
var = 500 # Increased variance for more noise
sigma = var**0.5
gauss = np.random.normal(mean, sigma, low_light_img.shape).astype('uint8')
low_light_img = cv2.add(low_light_img, gauss) # Add noise to the darkened image


# --- Basic Post-Processing Attempt --- #

# 1. Denoise using Non-local Means Denoising (a common technique for photographic noise)
# Parameters need tuning based on noise level
denoisied_img = cv2.fastNlMeansDenoisingColored(low_light_img, None, 10, 10, 7, 21)

# 2. Adjust brightness/contrast (simple gamma correction or linear adjustment)
# Here, we'll try to brighten it up linearly
brightened_denoised_img = cv2.convertScaleAbs(denoisied_img, alpha=1.5, beta=0) # alpha > 1 increases brightness

# --- Display Results ---
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
plt.title('Original Image (Simulated Scene)')
plt.axis('off')

plt.subplot(2, 2, 2)
plt.imshow(cv2.cvtColor(low_light_img, cv2.COLOR_BGR2RGB))
plt.title('Simulated Low-Light Capture (Dark & Noisy)')
plt.axis('off')

plt.subplot(2, 2, 3)
plt.imshow(cv2.cvtColor(denoisied_img, cv2.COLOR_BGR2RGB))
plt.title('Post-Processed: Denoised')
plt.axis('off')

plt.subplot(2, 2, 4)
plt.imshow(cv2.cvtColor(brightened_denoised_img, cv2.COLOR_BGR2RGB))
plt.title('Post-Processed: Denoised & Brightened')
plt.axis('off')

plt.tight_layout()
plt.show()
```


Original Image (Simulated Scene)
Simulated Low-Light Capture (Dark & Noisy)
Post-Processed: Denoised
Post-Processed: Denoised & Brightened